Computer Organization and Architecture Laboratory

MiniRISC Assignment 3: Fast Booth Multiplier Integration and Multi-Cycle Execution

Indian Institute of Technology Kharagpur
Department of Computer Science and Engineering

Laboratory Dates: 26 October and 28 October 2026

Objective

Extend the baseline MiniRISC processor completed up to 14 October by integrating the **previously implemented signed 32-bit fast Booth multiplier** as a processor execution unit.

This is a **two-day assignment**.

- **26 October:** integrate the multiplier into the MiniRISC datapath and control.
- **28 October:** complete the multi-cycle execution behavior and run a short multiplication-based program.

The Booth multiplier itself must **not be redesigned**.

1. Required Multiplication Instruction

The processor must provide a multiplication instruction equivalent to:

```
MUL Rd, Rs1, Rs2
```

with:

$$P = \text{signed}(R[Rs1]) \times \text{signed}(R[Rs2]).$$

For the common minimum implementation:

$$\boxed{R[Rd] \leftarrow P[31:0]}$$

where P is the signed 64-bit output of the Booth multiplier.

If a group has already defined a different multiplication-result convention in its ISA, that convention may be retained provided that the processor implementation remains consistent with it.

2. Existing Multiplier Interface

Reuse the previously verified signed 32-bit Booth multiplier.

The processor-facing multiplier interface should provide functionality equivalent to:

```
input  wire               clk;
input  wire               reset;
input  wire               start;
input  wire signed [31:0] multiplicand;
input  wire signed [31:0] multiplier;

output wire signed [63:0] product;
output wire               busy;
output wire               done;
```

The exact module and signal names may differ.

The Verilog multiplication operator `*` must not be used as a replacement for the previously designed Booth multiplier.

3. Required Datapath Integration

The two register-file read ports must provide the multiplier operands.

Conceptually:

$$R[Rs1] \rightarrow Booth\ Operand\ A$$

and

$$R[Rs2] \rightarrow Booth\ Operand\ B.$$

The write-back path must now support at least:

$$WriteData = \begin{cases} ALUResult, & \text{ALU instruction} \\ MemReadData, & \texttt{LD} \\ MulResult, & \texttt{MUL} \end{cases}$$

where:

$$MulResult = P[31:0].$$

Therefore, the existing write-back multiplexer must be extended to include the multiplier result.

4. Destination Register for a Multi-Cycle Instruction

The multiplication takes multiple clock cycles.

Therefore, the destination-register number of the `MUL` instruction must remain available until the product is ready.

For a 16-register processor, a register such as

```
reg [3:0] mul_dest_reg;
```

may be used.

When the multiplication starts:

$$mul_dest_reg \leftarrow Rd.$$

When the multiplier result becomes valid:

$$R[mul_dest_reg] \leftarrow P[31:0].$$

Equivalent implementations are acceptable.

5. Multiplier Operands

If required by the existing multiplier interface, the two multiplication operands may also be latched when `MUL` begins.

For example:

```
reg signed [31:0] mul_op_a;
reg signed [31:0] mul_op_b;
```

with:

$$mul_op_a \leftarrow R[Rs1]$$

and

$$mul_op_b \leftarrow R[Rs2].$$

If the earlier multiplier already captures both operands internally when `start` is asserted, additional operand registers are not required.

6. Required start, busy and done Behavior

start

`start` must initiate one multiplication.

It should normally be asserted for only one clock cycle for each `MUL` instruction.

The same multiplication must not be repeatedly restarted.

busy

While `busy`=1, the multiplier is processing the current operands.

A new multiplication must not be issued to the same multiplier during this interval.

done

When `done`=1, the multiplication result is available according to the multiplier interface.
The result must then be written into the stored destination register.

7. Signed Multiplication

The multiplication must operate on signed 32-bit values.
The following cases must work correctly:

$$7 \times 9 = 63,$$

$$(-7) \times 6 = -42,$$

and

$$(-4) \times (-5) = 20.$$

For example:

$$-42 = 32'hFFFFFFD6$$

when the low 32 bits are written into the destination register.

8. Multi-Cycle Processor Behavior

Each group may use either of the following two methods.

Method A: Hardware Stall

The processor waits for the multiplication to finish.
After `MUL` starts:

- hold the current instruction execution until the multiplier completes;
- prevent later dependent instructions from executing;
- hold the PC, or otherwise prevent issue of the next instruction;
- wait for `done`;
- write the result back;
- resume normal execution.

Conceptually:

$$MUL \rightarrow WAIT \rightarrow WAIT \rightarrow \cdots \rightarrow WRITEBACK \rightarrow NEXT.$$

A state such as:

```
MUL_WAIT
```

may be added to the control FSM.

Method B: Fixed-Latency NOP Scheduling

Alternatively, the processor may continue advancing the instruction stream if the multiplication latency is fixed.

In that case, sufficient `NOP` instructions must be placed between `MUL` and the first instruction that uses the result.

For example:

```
MUL R6, R2, R3
NOP
NOP
ADD R7, R7, R6
```

may be used if the processor implementation requires two intervening instruction slots.

The number of NOPs must match the actual implemented latency.

9. Required Processor Behavior

The implementation must ensure that:

1. the correct source registers are supplied to the multiplier;
2. `start` is issued exactly as required;
3. no second multiplication is started while the current multiplication is active;
4. the destination-register number is preserved;
5. the correct product is selected by the write-back path;
6. the result is not written before it is valid;
7. a dependent instruction does not use the result too early;
8. signed multiplication works correctly.

10. First-Day Demonstration: 26 October

By the end of 26 October, demonstrate the integrated multiplier using at least:

Test 1

$$7 \times 9 = 63.$$

Expected destination-register value:

$$32'h0000003F.$$

Test 2

$$(-7) \times 6 = -42.$$

Expected destination-register value:

$$32'hFFFFFFD6.$$

Test 3

$$(-4) \times (-5) = 20.$$

Expected destination-register value:

$$32'h00000014.$$

At least one case must begin from an actual decoded `MUL` instruction and end with register-file write-back.

11. Second-Day Program: Dot Product

On 28 October, execute a short processor program to calculate:

$$A = [2, -1, 3]$$

and

$$B = [4, 5, 6].$$

The required dot product is:

$$A \cdot B = (2)(4) + (-1)(5) + (3)(6)$$

$$= 8 - 5 + 18$$

$$\boxed{= 21}.$$

Therefore, the final 32-bit result is:

$$\boxed{32'h00000015}.$$

12. Data BRAM Contents

Use data BRAM to store the vectors.

A suggested word-addressed layout is:

Word Address	Value	Meaning
0	2	$A[0]$
1	-1	$A[1]$
2	3	$A[2]$
4	4	$B[0]$
5	5	$B[1]$
6	6	$B[2]$
8	0	Result location

Represent:

$$-1 = 32'hFFFFFFFF.$$

Groups may use another memory layout if the same computation is performed.

13. Required Dot-Product Operations

The program must perform:

$$ACC \leftarrow 0,$$

$$ACC \leftarrow ACC + A[0]B[0],$$

$$ACC \leftarrow ACC + A[1]B[1],$$

$$ACC \leftarrow ACC + A[2]B[2].$$

Finally:

$$Mem[8] \leftarrow ACC.$$

The final value stored in data BRAM must be:

$$\boxed{Mem[8] = 21}.$$

14. Example Program Structure

The program may be fully unrolled.

Conceptually:

```
ACC = 0

LD   RA, A0
LD   RB, B0
MUL  RT, RA, RB
[wait or required NOPs]
ADD  ACC, ACC, RT

LD   RA, A1
LD   RB, B1
MUL  RT, RA, RB
[wait or required NOPs]
ADD  ACC, ACC, RT

LD   RA, A2
LD   RB, B2
MUL  RT, RA, RB
[wait or required NOPs]
ADD  ACC, ACC, RT

ST   ACC, RESULT
HALT
```

The exact syntax depends on the group's ISA.

A loop-based implementation is also acceptable.

15. MAC Instruction

If a group has already implemented a dedicated `MAC` instruction, it may use it for the dot product.

However, `MUL` followed by `ADD` is sufficient for the assignment.

A separate MAC instruction is **not mandatory**.

16. Final Demonstration Requirements

By the end of 28 October, demonstrate:

1. correct signed `MUL`;
2. correct `start/busy/done` behavior;
3. correct destination-register write-back;
4. correct hardware-stall or NOP-scheduled operation;
5. correct dependent use of the multiplication result;
6. execution of the dot-product program;
7. final result

   21

   stored in data BRAM;

8. clean termination using `HALT`.

17. Files to Submit

Submit:

- complete updated MiniRISC Verilog source files;
- Booth multiplier integration/wrapper source;
- behavioral testbench;
- instruction BRAM initialization file;
- data BRAM initialization file;
- behavioral simulation waveform(s).

No redesign of the Booth multiplier, matrix multiplication, convolution, or UART implementation is required in this assignment.